1. Black box testing

1.1 Start Parking Session

Being able to create a new parking session is a crucial component of our application, needing to process carpark data for users to choose and edit their desired carpark location, indicate the relevant car that they want to create a session for, as well as allowing users to be able to input a session name, an optional session description, rate threshold and optionally multiple photos.

Thus, when creating a new parking session, there are certain preconditions and input data restrictions that need to be adhered to. Thus, precondition and input data validation is implemented when creating a new parking session to ensure correctness and completeness. Firstly, equivalence classes are partitioned by splitting a set of all possible inputs into two or more smaller subsets, where inputs of the same subset share some trait in common: having the same behaviour, producing the same output or when one input produces a bug, others of the same subset would likely produce the same bug. For some test cases, boundary values are tested to check how the system behaves at the limits of acceptable input values

Black box testing was done manually on our application with corresponding inputs, and outputs were recorded as observed.

1.2 Equivalence class & boundary value testing

Here lists the valid equivalence classes for each input when starting a parking session, excluding the optional inputs where all inputs are valid and accepted, thus are expected to behave in the same way. This will be the basis of creating our black box test cases.

1. Car Availability

Valid Equivalence Class	Invalid Equivalence Class
Has at least 1 car available before session creation	Has 0 cars available before session creation

2. Carpark location chosen

Valid Equivalence Class	Invalid Equivalence Class
A carpark location has been chosen on the map	A carpark location has not been chosen

3. Active session

Valid Equivalence Class	Invalid Equivalence Class
Has no active parking sessions	Has an active parking session

4. Session Name Field

Valid Equivalence Class	Invalid Equivalence Class
Input has >= 1 character in the Session Name field	Input has 0 characters in the Session Name field

5. Input Rate Threshold Field

Valid Equivalence Class (boundary)	Valid Equivalence Class	Invalid Equivalence Class	Invalid Equivalence Class
Input has 0 characters in the Input Rate Threshold field	Input is a non-negative numeric value in the Input Rate Threshold field	Input is non-numeric in the Input Rate Threshold field	Input is a negative numeric value in the Input Rate Threshold field

1.3 Test cases and Results

Test Case	Has >=1 Car	Carpark location is chosen	Active Session	Session Name Field	Input Rate Threshold Field	Expected Output	Actual Output
TC-01	Yes (Valid)	Yes (Valid)	No (Valid)	“Lunch Parking” (Valid)	3.50 (Valid)	Parking session successfully created	As expected, pass
TC-02	**No (Invalid)**	Yes (Valid)	No (Valid)	“Lunch Parking” (Valid)	3.50 (Valid)	"Create session" button is disabled.	As expected, pass
TC-03	Yes (Valid)	**No (Invalid)**	No (Valid)	“Lunch Parking” (Valid)	3.50 (Valid)	"Create session" button is disabled.	
TC-04	Yes (Valid)	Yes (Valid)	**Yes (Invalid)**	“Lunch Parking” (Valid)	3.50 (Valid)	Displays error message "Error: Could not create session." shown	
TC-05	Yes (Valid)	Yes (Valid)	No (Valid)	**"" (Invalid)**	3.50 (Valid)	"Create session" button is disabled.	
TC-06	Yes (Valid)	Yes (Valid)	No (Valid)	“Lunch Parking” (Valid)	**"abc" (Invalid)**	Displays error message “Error: Input a positive numeric number for input threshold”	

TC-07	Yes (Valid)	Yes (Valid)	No (Valid)	“Lunch Parking” (Valid)	**-3.50 (Invalid)**	Displays error message “Error: Input a positive numeric number for input threshold”	
TC-08	Yes (Valid)	Yes (Valid)	No (Valid)	“Lunch Parking” (Valid)	**"" (Valid, boundary)**	Session created without rate alert	As expected, pass

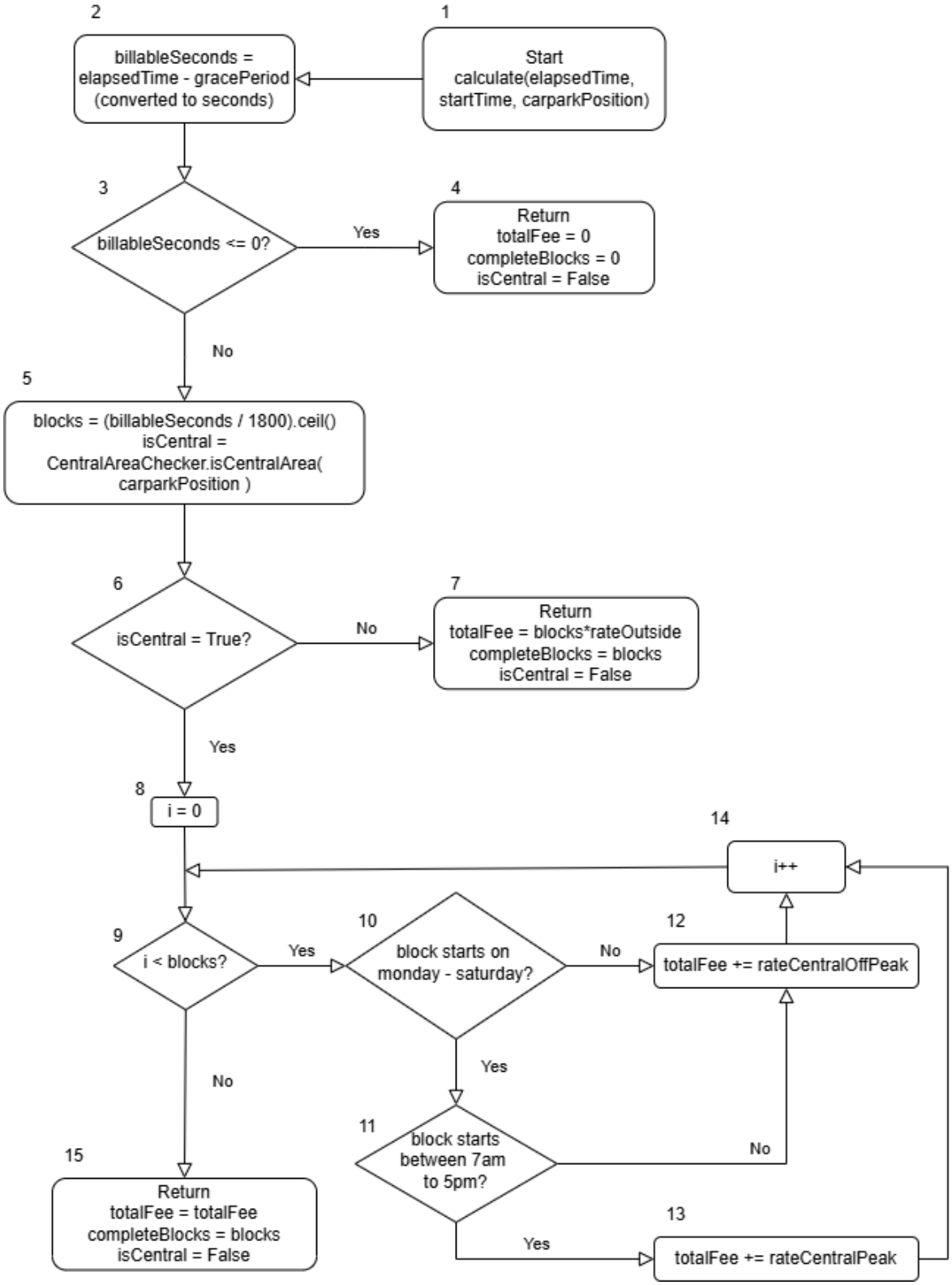
2. White box testing

2.1 Calculate

The calculate method is used to calculate the accumulated parking fee of that parking session. As the carparks retrieved from the API are all HDB carparks, this calculation is based on short-term parking charges for HDB carparks, $0.60 per half-hour for carparks outside the central area of singapore, $1.20 per half-hour for carparks within Central Area during peak hours (7:00am to 5:00pm, Monday to Saturday), and $0.60 per half-hour for carparks within Central Area during non-peak hours.

The code for the calculate method in hdb_fee_calculator.dart was tested with white box testing methods, ensuring that the inputs lead to expected and accurate outputs. Test code in white_box_tc_hdbcalc.dart was written to assist with the process of testing the method, using mocked data objects to simulate specific entry times and carpark locations, which allowed for the immediate verification of pricing calculations without needing to wait for real-world time to elapse or being restricted by the 1-kilometer carpark proximity filter implemented in the application.

2.1.1 Control Flow Graph



2.1.2 Basic Path Testing

Cyclomatic Complexity = | decision points | +1
= 5 + 1
= 6

Basis Paths

1. Baseline path TC-01: 1, 2, 3, 5, 6, 8, 9, 10, 11, 14, 9, … 9, 15
2. Basis Path TC-02: 1, 2, 3, 4
3. Basis Path TC-03: 1, 2, 3, 5, 6, 7
4. Basis Path TC-04: 1, 2, 3, 5, 6, 8, 9, 10, 12, 14, 9, … 9, 15
5. Basis Path TC-05: 1, 2, 3, 5, 6, 8, 9, 10, 11, 12, 14, 9, … 9, 15
6. Basis Path 06: 1, 2, 3, 5, 6, 8, 9, 15 (infeasible path)

Basis path 06 is an infeasible path since if we look at the logic, the requirements for the code to enter Basis path 06 is already covered by Basis Path TC-02. Thus, these two paths are mutually exclusive of each other, and the code will enter Basis Path TC-02 before ever reaching Basis Path 06.

Test Case	Description	Input	Expected result	Actual result
TC-01	Parking in central area during peak hours, charge $1.20 per half-hour block	startTime: on a Monday, 10:00 elapsedTime: 60 minutes carparkPosition: centralPosition	isCentral = True completedBlocks = 2 totalFee = 2.40	As expected, pass
TC-02	Billable seconds <= 0, parking was within grace period, no fee charged	startTime: on a Monday, 10:00 elapsedTime: 10 minutes carparkPosition: centralPosition	isCentral = False completedBlocks = 0 totalFee = 0	As expected, pass
TC-03	Parking not in central area, charge $0.60 per half-hour block	startTime: on a Monday, 10:00 elapsedTime: 60 minutes carparkPosition: nonCentralPosition	isCentral = False completedBlocks = 2 totalFee = 1.20	As expected, pass
TC-04	Parking in central area not from Monday-Saturday, charge $0.60 per half-hour block	startTime: on a Sunday, 10:00 elapsedTime: 60 minutes carparkPosition: centralPosition	isCentral = True completedBlocks = 2 totalFee = 1.20	As expected, pass
TC-05	Parking in central area from Monday-Saturday	startTime: on a Monday, 20:00 elapsedTime: 60 minutes	isCentral = True completedBlocks = 2 totalFee = 1.20	As expected, pass

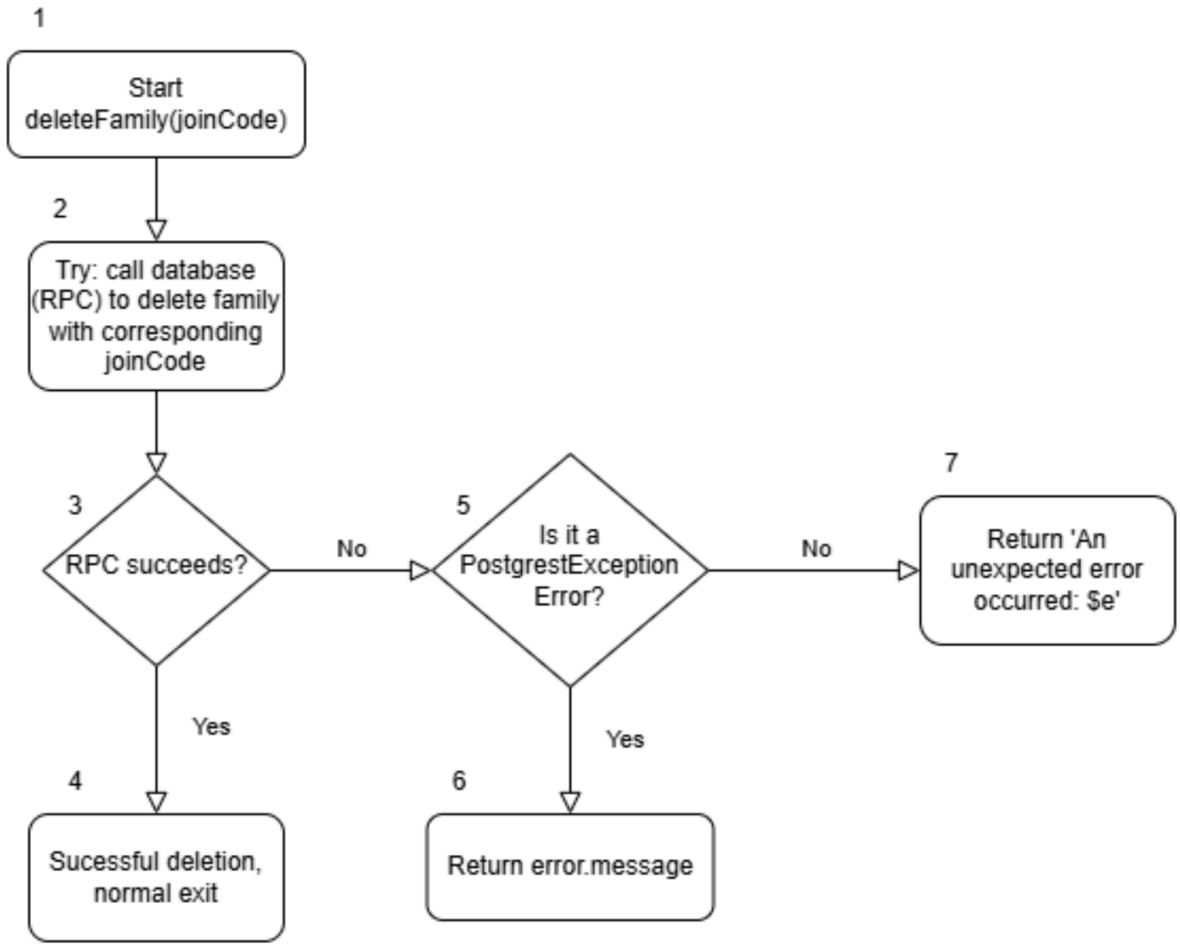
	but not between 7am-5pm, charge $0.60 per half-hour block	carparkPosition: centralPosition		

2.2 Delete Family

The deleteFamily method is used to delete an existing family from the database. As the code tries calling the database in Supabase to delete the family with the corresponding joinCode, we check if the family was successfully deleted. In the case that an error is raised, we make sure to return an appropriate error message.

The code for the deleteFamily method in family_service.dart was tested with white box testing methods, ensuring that the inputs lead to expected and accurate outputs. Test code in white_box_tc_family.dart was written to assist with the process of testing the method, simulating the errors that are hard to trigger manually such as database connection failures, verifying that the service handles exceptions gracefully without crashing the application.

2.2.1 Control Flow Graph



2.2.2 Basic Path Testing

Cyclomatic Complexity = | decision points | +1
= 2 + 1
= 3

Basis Paths

7. Baseline path TC-01: 1, 2, 3, 4
8. Basis Path TC-02: 1, 2, 3, 5, 6
9. Basis Path TC-03: 1, 2, 3, 5, 7

Test Case	Description	Input	Expected result	Actual result
TC-01	RPC completes without error and method returns successfully.	joinCode = "FAM-001" mock RPC → returns null (success)	Method completes, no exception is thrown	As expected, pass
TC-02	RPC throws a PostgrestException. The first catch block executes and rethrows only the .message string.	joinCode = "FAM-001" mock RPC → throws PostgrestException(message: "foreign key violation")	Throws String = "foreign key violation"	As expected, pass
TC-03	RPC throws a non-Postgrest exception (e.g. network error). The first catch does not match; the generic catch executes and rethrows with the "An unexpected error occurred:" prefix.	joinCode = "FAM-001" mock RPC → throws Exception("network timeout")	Throws String starting with "An unexpected error occurred:"	As expected, pass